

Révisions – TP3 RPOO

Table d'associations simple (AssocTable) — Master 1 Info, A. Plantec

1. Comprendre l'interface Map (Java)

Map<K,V> représente une association clé → valeur, sans clé dupliquée. Méthodes clés à connaître avant de coder :

- put(K key, V value) : ajoute ou remplace la valeur associée à une clé
- get(Object key) : renvoie la valeur associée à une clé (ou null)
- remove(Object key) : supprime le couple correspondant à une clé
- containsKey(Object key) / containsValue(Object value)
- size(), isEmpty()
- keySet(), values(), entrySet() — vues sur les clés/valeurs/couples

Map.Entry<K,V> : représente un couple (clé, valeur) à l'intérieur de la map, avec getKey() et getValue().

2. Structure de données : liste de couples

Le TP demande de stocker **assocs** comme une liste (ArrayList ou List) de couples clé-valeur. Il faut donc une classe (interne ou séparée) représentant un couple :

```
public class Couple<K,V> implements Map.Entry<K,V> {
    private K cle;
    private V valeur;

    public Couple(K cle, V valeur) {
        this.cle = cle;
        this.valeur = valeur;
    }

    public K getKey() { return cle; }
    public V getValue() { return valeur; }
    public V setValue(V nouvelleValeur) {
        V ancienne = valeur;
        valeur = nouvelleValeur;
        return ancienne;
    }
}
```

3. Révisions Java indispensables

- **Générique (<K,V>)** : comment déclarer et utiliser une classe générique.
- **List<T> et ArrayList<T>** : add(), remove(), get(int i), size(), parcours avec boucle for classique ou for-each.
- **Itérateurs (Iterator)** : comment les utiliser, et surtout comment supprimer un élément pendant l'itération sans ConcurrentModificationException (utiliser it.remove() et non list.remove() directement dans la boucle).
- **equals()** : pour comparer deux clés, utiliser cle.equals(autreCle) et non == (sauf cas particulier des types primitifs auto-boxés).

- **Exceptions** : `NullPointerException` si on essaie d'insérer une clé null (l'énoncé précise qu'une clé ne peut jamais être null → il faut lever une exception ou refuser l'opération).

4. Logique de l'algorithme put

D'après le schéma du sujet : si la clé existe déjà → on remplace la valeur (pas de doublon de clé). Si la clé n'existe pas → on ajoute un nouveau couple à la fin de la liste.

```
put(cle, valeur):
    pour chaque couple dans assocs:
        si couple.getKey().equals(cle):
            ancienne = couple.getValue()
            couple.setValue(valeur)
            retourner ancienne
    assocs.add(new Couple(cle, valeur))
    retourner null
```

5. Logique de remove

```
remove(cle):
    pour chaque couple dans assocs (avec itérateur) :
        si couple.getKey().equals(cle):
            valeur = couple.getValue()
            it.remove()
            retourner valeur
    retourner null
```

6. Logique de get

```
get(cle):
    pour chaque couple dans assocs:
        si couple.getKey().equals(cle):
            retourner couple.getValue()
    retourner null
```

7. Point de vigilance sur entrySet()

Cette méthode doit renvoyer un `Set<Map.Entry<K,V>>` pour permettre l'itération montrée en annexe (`printMap`). Le plus simple : renvoyer un `Set` construit à partir de votre liste `assocs` (ex. `new HashSet<>(assocs)`), en s'assurant que votre classe `Couple` implémente bien `Map.Entry`.

8. Plan de test à préparer soi-même

Reproduisez l'exemple du schéma du sujet :

```
AssocTable<String,Integer> m = new AssocTable<>();
m.put("G", 2);
m.put("A", 7);
m.put("A", 3);    // doit remplacer 7 par 3, pas dupliquer
m.put("X", 5);
m.remove("A");
printMap(m);      // doit afficher G=2 et X=5
```